

DataLens — References and Prior Art

Darwinbox FDE take-home · Natural-language analytics over uploaded CSV/XLSX files · September 2026

1. Scope of this document

DataLens contains no code copied from any paper, repository, or third-party library beyond its declared npm dependencies (Next.js, the OpenAI SDK used as an OpenRouter client, `xlsx/csv` parsers, charting). Every module under `src/lib/` was written for this project against its own types. What the system does borrow are *techniques* — design ideas from published research and from established business-intelligence practice. This document lists those ideas, where each one lives in the codebase, and the work it is drawn from, so the attribution is explicit.

2. Techniques and their sources

Technique in DataLens	Where it lives	Drawn from
Multi-fact aggregation. When a question needs measures from two fact tables (e.g. an invoices table and a payments table), each table is aggregated separately at the requested grain and the results are merged on the shared dimensions. Joining the two fact tables first would multiply rows (the “chasm trap” / “fan trap”) and inflate every total.	<code>compile-selection.ts</code> (one sub-plan per fact table), <code>merge-results.ts</code> (full-outer merge on dimension values)	Looker “symmetric aggregates”; dbt MetricFlow join rendering (“avoids fan-out and chasm joins”); Sisense documentation on chasm and fan traps [1–3]
Inferred semantic layer. Tables, measures, dimensions and label references are inferred from the uploaded files and rendered as a compact menu. The model selects fields from the menu; the compiler derives the base table, join path, post-join column names, anti-joins and derived columns deterministically.	<code>semantic-model.ts</code> , <code>compile-selection.ts</code> , <code>llm.ts</code> (SELECTION_SYSTEM prompt)	dbt Semantic Layer benchmark (LLM accuracy on analytical questions rose from 16% to 83% when a semantic layer sat between the model and the warehouse) [4]; write-ups on reducing the “constraint tax” of large tool schemas for small models [5]
Primary/foreign-key inference. A relationship between two files is proposed only when one side is (near-)unique and the other side's values are included in it (an inclusion dependency), with cardinality classified as 1:1, 1:N, N:1 or N:M. A key-ness filter (a unique side, or at least 20 distinct values on both sides) prevents low-cardinality columns that merely share a vocabulary from linking every table to every other.	<code>relationships.ts</code> , <code>keys.ts</code>	Inclusion-dependency discovery (Papenbrock et al., Metanome line of work) [6]; Auto-BI join-relationship detection (Microsoft Research) [7]
Schema linking and pruning. Question words are matched to columns (stemming, plurals, abbreviations; strong vs weak links), join paths are found by breadth-first search over inferred relationships, and only the linked tables plus one hop are shown in full to the model — the rest as one-liners. This keeps the prompt small enough for free-tier open-source models at 25 files.	<code>schema-linking.ts</code> , <code>buildPlannerUserMessage</code> in <code>llm.ts</code>	DCG-SQL (schema-linking for text-to-SQL); Lacuna (schema pruning); SchemaGraphSQL (graph-based join-path selection) [8–10]
Structured plan → validate → repair → retry. The model returns a structured plan (or selection) rather than free-form SQL. A deterministic validator repairs it, an answer-check assesses whether it can answer the question's shape, and the model is re-asked with concrete feedback up to three times. If no model is reachable, no answer is produced.	<code>plan-validator.ts</code> , <code>answer-check.ts</code> , <code>app/api/query/route.ts</code>	The plan-verify-repair pattern of DIN-SQL / MAC-SQL style text-to-SQL pipelines [11, 12]
Chart-type selection. Rules mapping question shapes and result shapes (categories, time, proportions, distributions, two numeric columns) to chart types.	<code>chart-eval/reference.ts</code> (each rule carries a <code>source</code> : URL inline)	Data-visualisation guidance from ThoughtSpot, Atlassian, Domo, Luzmo, Inforiver, Yurbi, Querio [13]

3. What is original to this project

- The combination of the above into one deterministic pipeline that runs on free open-source models via OpenRouter, with a full step-by-step trace exposed in the UI.
- Arithmetic verification of spreadsheet summary rows: a row labelled “Total” (or similar) is excluded only if each of its numeric cells equals the sum of every other row — so a genuine entity that happens to be named “Total ...” is never dropped.
- The answer-check rules (aggregate expected but absent, formula spelled out but not derived, material repairs, unit-rate × quantity “value” questions, unreachable linked columns) and the reasoning-preserving retry when a model runs out of output room mid-derivation.
- The rate-limit strategy for a chain of free models across several API keys: daily-cap skip, congestion cool-down, a wall-clock budget per planner call, and skipping delisted models rather than aborting.
- All offline regression suites (schema linking, selection compiler, answer check, relationship inference, validator, adversarial fixtures, 25-file scale benchmark) and the deterministic generator for the interdependent validation data set.

4. References

- [1] Looker documentation, “Symmetric aggregates” — <https://cloud.google.com/looker/docs/best-practices/understanding-symmetric-aggregates>
- [2] dbt Labs, “How the dbt Semantic Layer works” (MetricFlow join rendering) — <https://www.getdbt.com/blog/how-the-dbt-semantic-layer-works>
- [3] Sisense documentation, “Chasm and fan traps” — <https://docs.sisense.com/main/SisenseLinux/chasm-and-fan-traps.htm>
- [4] dbt Labs, “Semantic Layer benchmark: LLM accuracy with and without a semantic layer” — <https://www.getdbt.com/blog/semantic-layer-as-the-data-interface-for-llms>
- [5] Community write-ups on adapting tool/JSON schemas to small open-weight models (“the constraint tax”); consulted for prompt design, no code used.
- [6] T. Papenbrock et al., “Divide & Conquer-based Inclusion Dependency Discovery”, PVLDB 8(7), 2015; and the Metanome data-profiling platform.
- [7] Y. He et al., “Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph”, PVLDB 16(10), 2023.
- [8] DCG-SQL: “Dynamic Context Generation for Text-to-SQL” — schema linking via question–column matching (arXiv, 2024).
- [9] Lacuna: schema pruning for large-schema text-to-SQL with small models (arXiv, 2025).
- [10] SchemaGraphSQL: graph-based join-path selection for multi-table text-to-SQL (arXiv, 2025).
- [11] M. Pourreza & D. Rafiei, “DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction”, NeurIPS 2023.
- [12] B. Wang et al., “MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL”, COLING 2025.
- [13] Chart guidance pages cited inline in `src/lib/chart-eval/reference.ts` (ThoughtSpot, Atlassian, Domo, Luzmo, Inforiver, Yurbi, Querio).

Model access: all language-model calls go through OpenRouter (<https://openrouter.ai>) to free-tier open-source models (Qwen 3.8 27B, Gemma 4, GLM 5.2, NVIDIA Nemotron 3, Nex N2.5, Inkling). No paid model is used.